Project Proposal and Methodology

Hybrid RAG & Fine-Tuning for Customer Support · Stage 1 deliverable (Tasks 1.1 and 1.4)

1. Business Objective

An online store's first-line support gets a steady stream of repetitive questions: where an order is, how refunds work, why a card was charged, how to reset a password. A general-purpose chat model can answer these in fluent English, but it doesn't know *this* company's rules. It makes up refund windows, delivery timelines and contact channels, and it often misreads what the customer actually wants when the message is angry, sarcastic or vague.

The goal of this project is a small, open-source support assistant that (a) works out the customer's intent reliably, even from messy text, and (b) answers only from the company's Standard Operating Procedures (SOPs). We build it in three steps so the contribution of each component can be measured separately:

1. **Baseline** – the pre-trained instruct model answers on its own.
2. **Solution V1 (Naive RAG)** – the raw customer message is used to retrieve an SOP, which is injected into the prompt.
3. **Solution V2 (Hybrid RAG)** – a LoRA fine-tuned router first turns the message into a strict JSON intent; that intent, not the noisy text, drives retrieval.

2. Task 1.1 — Project Scope

2.1 Target use case (Subtask 1.1.1)

The selected support category is **post-purchase e-commerce support**, i.e. tickets raised after a customer has placed an order: order status and delivery delays, refunds and returns, payments and invoices, and the account access needed to manage those orders. The running example used across all notebooks is a delivery-delay complaint:

```
"ugh my order STILL hasn't arrived... how many days is delivery even supposed to take??"
```

The ground truth for this query comes from `shipping_delays.md`: domestic orders deliver in 3–7 business days, international in 10–21, and an order only counts as delayed once it passes the upper bound.

The router is trained on the full Bitext intent taxonomy (27 intents, 11 categories). Keeping the neighbouring intents is deliberate. A router that has only seen shipping questions can't tell a delivery complaint from a refund request, and that confusion is exactly the failure we want to fix.

2.2 Inputs and outputs

Component	Input	Output
Intent router (V2)	Raw customer message (free text)	Strict JSON only: `{"intent": "delivery_period", "category": "DELIVERY"}`

Retriever	V1: raw message. V2: the search string stored for the predicted intent (the full text of the SOP that intent maps to)	Top-1 SOP document (ChromaDB, cosine)
Generator	System prompt with the SOP + customer message	Short customer-facing answer grounded in the SOP

JSON rules: exactly two keys in fixed order; `intent` is lowercase snake_case from the closed catalog; `category` is UPPERCASE; nothing is allowed before or after the object.

2.3 Boundaries

- **In scope:** single-turn English messages; intent classification; retrieval over 13 Markdown SOPs; SOP-grounded answer generation; offline evaluation on a held-out split.
- **Out of scope:** multi-turn dialogue memory, real order/CRM lookups, taking actions (issuing refunds, resetting passwords), non-English input, production serving and latency SLAs.
- **Constraints:** free Google Colab T4 (16 GB), open-source models only, 1,000–5,000 training records.

2.4 Success criteria (Subtask 1.1.2)

These targets are fixed before any development so the results can be judged against them rather than rationalised afterwards.

Metric	Definition	Target
Router Format Adherence	% of router outputs where the whole string is valid JSON with both keys	≥ 95%
Intent Accuracy (exact)	% of predicted intents equal to the gold intent, held-out test split	≥ 85%, and ≥ 80% on the adversarial subset
Retrieval Hit@1	% of queries where the retrieved SOP equals the intent-mapped SOP	V2 ≥ 90% and clearly above V1
ROUGE-1 / ROUGE-L / BLEU	Similarity of the final answer to the SOP-grounded reference	V2 > V1 > Baseline
Hallucination Frequency	% of answers with an unsupported time/amount, a placeholder or invented contact detail, or a request for secrets	≥ 50% relative reduction, Baseline → V2
Consistency Rate	% of queries giving identical output over 3 deterministic runs	100% for all systems

3. Data

- **Instruction data:** `bitext/Bitext-customer-support-llm-chatbot-training-dataset` (~26.9k rows; columns `instruction`, `intent`, `category`, `response`). A 4,000-row random sample (`random_state=42`) keeps about 148 examples per intent and keeps training within a T4 session.
- **Knowledge base:** 13 SOPs from `Dataset.zip`, including the five required ones (refund policy, shipping delays, password reset, technical troubleshooting, escalation matrix), plus account recovery, billing disputes, data privacy, order tracking, payment methods, product return, subscription cancellation and working hours. Each file is about 1.5–1.8k characters with an H1 title and several H2 sections.

- **Quality checks (EDA notebook):** null and empty-field checks, formatting checks (whitespace, casing, Bitext `{{placeholders}}`), exact and normalised (lowercase + strip) duplicate detection, intent→category one-to-one alignment, class balance, token-length distribution, and TF-IDF cosine similarity between SOPs to spot overlapping or conflicting rules.
- **Partitions:** stratified 80/10/10 split on `intent`. Leakage is checked with set intersections on normalised prompts across all three pairs. The validation split is used only for loss monitoring and early stopping. The test split is used only for final scoring, and the adversarial subset is regex-filtered out of it.

4. Task 1.4 — Methodology

```
customer message → [LoRA router] → {"intent","category"} → intent→SOP search string →
                [ChromaDB top-1] → SOP → [Qwen2.5 generator] → answer
 (V1 skips the router and searches with the raw message; the Baseline skips retrieval
                                     entirely)
```

4.1 Baseline model (Subtask 1.4.1)

Choice: **`Qwen/Qwen2.5-1.5B-Instruct`**, used as the same `MODEL_ID` in every notebook.

- **Hardware:** loaded in 4-bit NF4 it needs roughly 1.2 GB of VRAM, which leaves room on a 16 GB T4 for QLoRA training with gradient checkpointing, and for holding the generator and the merged router together at evaluation time. Llama-3-8B would need more than 16 GB during fine-tuning.
- **Data:** Bitext queries are short (median about 10 tokens) and the JSON target is about 15 tokens, so a 1.5B model has plenty of capacity. The ChatML template is native to Qwen, so no custom tokens are needed.
- **Fine-tuning feasibility:** Qwen2.5 follows instructions and emits structured output better than TinyLlama-1.1B, and one epoch over ~3.1k examples takes well under an hour on a T4.

Inference is deterministic in every system: `do_sample=False` with `temperature=None, top_p=None, top_k=None` (never `temperature=0.0`), `max_new_tokens=120` for answers and 40 for the router.

4.2 Retrieval strategy (Subtask 1.4.2)

- **Loading and chunking:** LangChain `TextLoader`, then `RecursiveCharacterTextSplitter` with a 2,000-character chunk size, zero overlap and `##` -header separators. Every SOP fits in a single chunk, so a rule and its exceptions are never split apart. If the corpus grows, the header separators allow section-level chunks.
- **Embeddings:** `sentence-transformers/all-MiniLM-L6-v2` (384-dim, L2-normalised), the same model in NB4, NB5 and NB7. Validation: shape (13, 384), finite values, unit norm, and a pairwise SOP similarity check.
- **Vector store:** persistent ChromaDB, collection `sop_policies`, `hnsw:space = cosine`. It is built on local disk and copied to Google Drive, because Chroma's SQLite file is unreliable directly on the Drive mount.
- **Similarity search:** top-k = 1, since each document is a whole focused policy and more documents would add conflicting rules to the prompt. V1 searches with the raw message. V2 maps the router's intent to a stored search string, the full text of the mapped SOP, and falls back to the raw message if the JSON can't be parsed. A bare title such as `"# Refund Policy"` is too short to embed reliably, and title plus first section is not enough either: the Scope sections of shipping

delays and order tracking are nearly identical. Since each SOP is a single chunk, its full text always retrieves itself.

- **Retrieval evaluation:** every intent's search string must retrieve its own SOP, plus Hit@1 on the full test split, where the gold SOP comes from the gold intent.

4.3 Fine-tuning strategy (Subtask 1.4.3)

- **Method:** QLoRA. The base model is frozen in 4-bit NF4 with double quantisation and fp16 compute, prepared with `prepare_model_for_kbit_training` and gradient checkpointing.
- **Adapters:** `r=16`, `lora_alpha=32`, `dropout=0.05`, target modules `q_proj` and `v_proj`, so well under 1% of parameters are trainable. We don't add `k_proj`/`o_proj`: the task only changes the output format and label choice, and extra capacity mostly raises the risk of overfitting on ~3k rows.
- **ChatML structure:**

```
<|im_start|>system
You are an intent router ... reply with ONLY a JSON object ...
Catalog (CATEGORY: intents): ...     + 4 hand-written few-shot examples
<|im_end|>
<|im_start|>user
{customer message}<|im_end|>
<|im_start|>assistant
{"intent": "track_order", "category": "ORDER"}<|im_end|>
```

- **Label masking:** every system, user and padding token is set to `−100`, so the loss covers only the JSON and its closing `<|im_end|>`. `max_length=384` is chosen from the token-length analysis so no target is truncated.
- **Training loop:** custom PyTorch loop with `AdamW` (lr 2e-4, weight decay 0.01), linear warm-up (20 steps) and decay, batch size 4, one epoch, gradient clipping at 1.0, seed 42.
- **Monitoring and checkpointing:** validation loss (token-weighted) every 25 steps. Whenever it improves, the adapter is saved to `intent_lora_best/`. Early stopping triggers after 3 checks without improvement. The final step is saved to `intent_lora/` as a backup.
- **Logging:** `training_log.csv` (step, train loss, val loss, lr, elapsed time), `training_curves.png`, and `training_config.json` (hyperparameters, library versions, GPU, best step).

4.4 Evaluation framework (Subtask 1.4.4)

All systems are scored on the same held-out test split, with the same generator, decoding settings, index and prompt template, so the only thing that changes between V1 and V2 is how the search query is built.

Metric	Procedure
Format Adherence	Router: `json.loads` on the full raw output must succeed and contain both keys (a regex rescue is used for routing but doesn't count as adherent). Free-text answers: non-empty (≥ 3 words) with no leaked chat-template or role tokens.
Intent Accuracy	Exact match of the predicted intent against the gold intent, on the test split and on the adversarial subset: test queries matching a regex for emotional (*still, never, awful, again*), rude (*damn, bloody* and stronger) or hedging (*maybe, not sure, I don't know, wanna*) wording. Fuzzy match (same token set or string similarity ≥ 0.8) is reported alongside it. The un-tuned base model with the same prompt is scored as a router too, to isolate what the adapter adds.
ROUGE / BLEU	ROUGE-1 and ROUGE-L F1 (stemmed) and smoothed sentence BLEU against the full text of the SOP that the gold intent maps to, so answers are rewarded for using the right policy's

	content rather than for sounding generic.
Consistency	20 fixed test queries × 3 runs per system; a query counts as consistent only if all 3 outputs are identical.
Hallucination Frequency	An answer is flagged if it states a time, amount or percentage whose numbers don't appear with a unit in the correct SOP, leaves a template placeholder (e.g. *[insert phone number]*) or invents a URL, email or phone number, or asks the customer for secrets (password, CVV, one-time code).

Improvement is reported as `(new − old) / old × 100` for Baseline→V1 (retrieval's contribution) and V1→V2 (the router's contribution).

5. Risks and Mitigations

Risk	Mitigation
Near-duplicate prompts leaking across splits	Normalised dedup before splitting; set-intersection asserts on normalised text
Router adds prose around the JSON	Only the JSON is supervised; short `max_new_tokens`; strict parsing with a regex fallback
Similar SOPs confused (refund / billing / return)	TF-IDF overlap analysis in EDA; intent-driven search strings in V2
Rule-based hallucination check misses paraphrased fabrications	Documented as a limitation; manual spot checks of flagged and unflagged samples
Colab session loss	All artifacts written to Google Drive after each notebook

6. Deliverables and Notebook Flow

Notebook	Produces
1 · Data Understanding & EDA	`sampled_data.csv`, `corporate_policies/`
2 · Data Preparation	`tokenized_train/valid/test`, `df_test.csv`, `router_config.json`, `intent_sop_map.json`
3 · Baseline Evaluation	`outputs.json`
4 · RAG Implementation	`chroma_db/`, updated `outputs.json`
5 · Solution V1 Evaluation	`v1_metrics.csv`, `v1_summary.csv`
6 · Fine-Tuning Pipeline	`intent_lora_best/`, `training_log.csv`, `training_curves.png`
7 · Solution V2 Evaluation	`Comparative_Results_Full.csv`, `Comparative_Results_Summary.csv`